from a browser. For this, we're going to use a library provided by the ASP.NET Core team called Microsoft.AspNetCore.TestHost.

36.3.1 Creating a TestServer using the Test Host package

Imagine you want to write some integration tests for the `StatusMiddleware` from section 36.1. You've already written unit tests for it, but you want to have at least one integration test that tests the middleware in the context of the ASP.NET Core infrastructure.

You could go about this in many ways. Perhaps the most complete approach would be to create a separate project and configure `StatusMiddleware` as the only middleware in the pipeline. You'd then need to run this project, wait for it to start up, send requests to it, and inspect the responses.

This would possibly make for a good test, but it would also require a lot of configuration, and it would be fragile and error-prone. What if the test app can't start because it tries to use an already-taken port? What if the test app doesn't shut down correctly? How long should the integration test wait for the app to start?

The ASP.NET Core Test Host package lets you get close to this setup without having the added complexity of spinning up a separate app. You add the Test Host to your test project by adding the Microsoft.AspNetCore.TestHost NuGet package, using the Visual Studio NuGet GUI, Package Manager Console, or .NET command-line interface (CLI). Alternatively, add the `<PackageReference>` element directly to your test project's .csproj file: